

Time Series Bootcamp

Why time order matters, and how to forecast

Monnie McGee

June 10, 2026

REU Bootcamp

Install once

```
install.packages("fpp3")  
install.packages("broom")
```

Load for this workshop

```
library(fpp3)  
library(broom)
```

Primary Reference

Hyndman, R.J. & Athanasopoulos, G. (2021), *Forecasting: Principles and Practice* (3rd ed.) <https://otexts.com/fpp3/>

Two-hour learning goals

By the end of this session, you should be able to:

1. Recognize when data are time series data.
2. Define autocorrelation.
3. Explain why ordinary regression can be misleading for autocorrelated data.
4. Identify trend, seasonality, cycles, and noise in a time plot.
5. Describe forecasting as a train/test prediction problem.
6. Fit and compare a few simple forecasting models in R.

Reference spirit: Hyndman and Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed.

1. Why ordinary regression often fails

Temperature Example

Suppose we record temperature every day.

Question:

If you know today's temperature, do you know something about tomorrow's temperature?

Temperature Example

Suppose we record temperature every day.

Question:

If you know today's temperature, do you know something about tomorrow's temperature?

This is **autocorrelation**: correlation between observations from the same series at different times.

Regression assumption being challenged

Ordinary least squares is often introduced with independent errors:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \varepsilon_i \text{ independent.}$$

For time series, observations arrive in order:

$$y_t, y_{t-1}, y_{t-2}, \dots$$

A common time-series feature is:

$$\text{Corr}(y_t, y_{t-1}) \neq 0.$$

The past carries information about the future.

Simulation: autocorrelated data with no trend

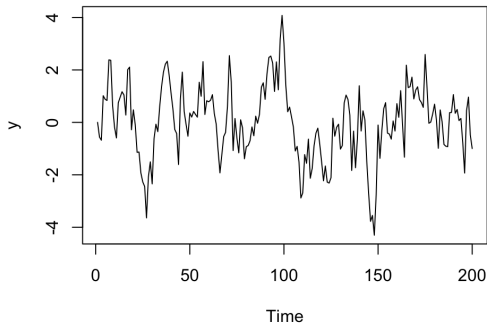
```
set.seed(123)

n <- 200
y <- numeric(n)

for(t in 2:n){
  y[t] <- 0.8 * y[t-1] + rnorm(1)
}

time <- 1:n
plot(time, y, type = "l",
     xlab = "Time", ylab = "y")

fit_lm <- lm(y ~ time)
summary(fit_lm)
```



The true process has no linear time trend, but it has long runs above and below zero. 5/100

Examine regression residuals

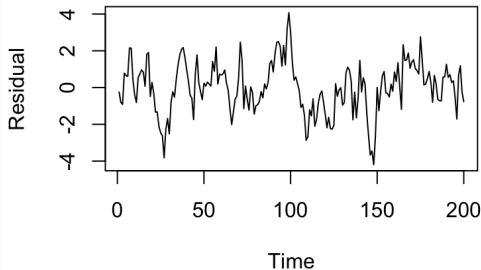
```
plot(residuals(fit_lm), type = "l",  
     xlab = "Time", ylab = "Residual")
```

Examine regression residuals

```
plot(residuals(fit_lm), type = "l",  
     xlab = "Time", ylab = "Residual")
```

Thought questions:

- Do residuals look independent?
- Are there clusters of positive or negative residuals?



Important nuance: what exactly goes wrong?

Autocorrelation alone does **not** automatically make OLS slopes biased.

If the regression model is correctly specified and regressors are exogenous, OLS coefficients can still be unbiased.

But ordinary regression can still be badly misleading because:

- standard errors and p-values can be wrong;
- trend or seasonality may be omitted;
- unrelated trending series can look strongly related;
- forecasts ignore useful information in recent observations.

Strange Example: Temperature and CO₂

Suppose we regress global temperature anomaly on atmospheric CO₂ concentration:

$$\text{Temperature}_t = \beta_0 + \beta_1 \text{CO}_{2,t} + \varepsilon_t.$$

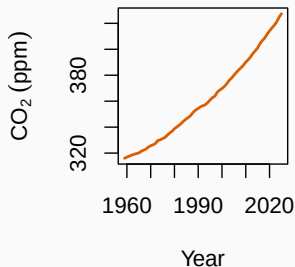
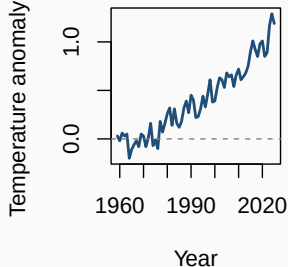
Two of the most authoritative and stable public datasets for this topic:

NASA GISTEMP v4 is a data set of global mean temperature anomalies relative to the 1951–1980 baseline, updated monthly through the present. The file is a CSV pulled directly from `data.giss.nasa.gov`.

NOAA GML Mauna Loa annual mean CO₂ the longest continuous atmospheric CO₂ record, available as a CSV from `gml.noaa.gov`.

Code to download and analyze data is in R Markdown File

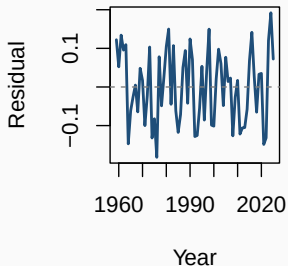
Global mean temperature anomaly, Mauna Loa
(GISTEMP v4, baseline 1951-1980) atmospheric CO₂ (NOAA GML, annual average)



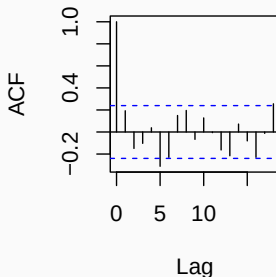
The strong increase in CO₂ is very close to the trend in temperature. A linear regression will “regress it out”, leaving almost white noise residuals.

White Noise Residuals

OLS residuals over time



ACF of OLS residuals



Because they share a common trend, but one series has a lot less noise than the other, a regression of temperature on CO_2 removes the trend, resulting in white noise residuals.

Consider two independent random walks:

$$x_t = x_{t-1} + u_t, \quad y_t = y_{t-1} + v_t.$$

The error terms u_t and v_t are statistically independent, so the two series are unrelated.

But if both series wander over time, a regression of y_t on x_t may still produce:

- a significant slope,
- a high R^2 ,
- autocorrelated residuals.

Your Turn: two independent random walks

```
set.seed(2026)

n <- 200
x <- cumsum(rnorm(n))
y <- cumsum(rnorm(n))
time <- 1:n

plot(time, x, type = "l", ylim = range(c(x, y)),
     xlab = "Time", ylab = "Value")
lines(time, y, lty = 2)
legend("topleft", legend = c("x", "y"), lty = c(1, 2), bty = "n")

fit_rw <- lm(y ~ x)
summary(fit_rw)
acf(residuals(fit_rw))
```

What do you see?

2. What makes time series different?

Four common time-series features

Trend Long-term movement upward or downward.

Seasonality Repeating pattern with a fixed calendar period.

Cycles Longer, less regular rises and falls.

Noise Random variation left after structure is modeled.

Other common examples

Series	Likely structure	Forecasting use
Daily temperature	seasonality, autocorrelation	weather planning
Monthly retail sales	trend, seasonality	inventory
Electricity demand	daily/weekly seasonality	grid management
Population	trend	planning services
Website traffic	trend, seasonality, shocks	staffing/content

Time-series methods are appropriate when the order of observations carries information.

3. Forecasting mindset

Regression mindset vs forecasting mindset

Regression mindset

- Estimate relationships.
- Interpret coefficients.
- Ask causal or explanatory questions.

Forecasting mindset

- Predict future observations.
- Evaluate accuracy on held-out data.
- A useful forecast need not be causal.

In forecasting, the future is the test set.

A basic forecasting task

1. Define what must be forecast.
2. Decide the forecast horizon.
3. Separate training data from future test data.
4. Fit candidate models on training data.
5. Compare forecasts to held-out observations.

This is the workflow emphasized in forecasting texts such as FPP3.

Forecasting by Hand

```
library(fpp3)

recent_beer <- aus_production |>
  filter(year(Quarter) >= 1992)

train <- recent_beer |>
  filter(year(Quarter) <= 2007)

test <- recent_beer |>
  filter(year(Quarter) > 2007)

train |>
  autoplot(Beer) +
  autolayer(test, Beer, color = "gray") +
  labs(title = "Hide the future first")
```

Sketch the next few quarters before fitting any model.

Simple forecasting methods

Three benchmark methods

Before complicated models, use simple baselines.

Mean forecast Future values equal the historical average.

Naive forecast Future values equal the most recent observation.

Seasonal naive Future values equal the value from the same season last year.

A model is only impressive if it beats good baselines.

Fit benchmark forecasts in fpp3

```
fit_benchmarks <- train |>
  model(
    Mean = MEAN(Beer),
    Naive = NAIVE(Beer),
    Seasonal_naive = SNAIVE(Beer)
  )

fc_benchmarks <- fit_benchmarks |>
  forecast(h = nrow(test))

fc_benchmarks |>
  autoplot(train, level = NULL) +
  autolayer(test, Beer) +
  labs(title = "Benchmark forecasts")
```

Compare forecast accuracy

```
accuracy(fc_benchmarks, test)
```

Useful metrics:

- RMSE: large errors are penalized heavily.
- MAE: average absolute error.
- MAPE: percentage error, but problematic near zero.

Keep the interpretation simple: lower error is better.

Exponential smoothing

Forecasting tomorrow's temperature

One simple forecast is the historical average:

$$\hat{y}_{t+1} = \frac{y_1 + y_2 + \cdots + y_t}{t}.$$

Thought Question:

Should a temperature from three years ago count as much as yesterday's temperature?

Makes sense to give more weight to recent observations.

Exponentially weighted averages

Exponential smoothing forecasts using weights that shrink as observations get older:

$$\hat{y}_{t+1} = \alpha y_t + \alpha(1 - \alpha)y_{t-1} + \alpha(1 - \alpha)^2 y_{t-2} + \cdots .$$

Main idea

Recent observations matter most, but older observations are not thrown away completely.

The recursive form

Simple exponential smoothing can be written as

$$\ell_t = \alpha y_t + (1 - \alpha)\ell_{t-1}.$$

Here ℓ_t is the current estimate of the level.

Plain-language interpretation

New estimate = weighted average of what we just observed and what we previously believed.

What does α do?

Value of α	Behavior	Interpretation
Small, such as 0.1	Very smooth	Slow to adapt
Medium, such as 0.5	Balanced	Moderate adaptation
Large, such as 0.9	Wiggly	Responds quickly

Key intuition

The smoothing parameter controls the tradeoff between stability and responsiveness.

Manual demonstration of α

```
ses_level <- function(y, alpha) {  
  level <- numeric(length(y))  
  level[1] <- y[1]  
  
  for (t in 2:length(y)) {  
    level[t] <- alpha*y[t] + (1 - alpha)*level[t - 1]  
  }  
  
  level  
}  
  
beer <- aus_production |>  
  filter_index("1990 Q1" ~ "2005 Q4") |>  
  select(Quarter, Beer)
```

Plot three smoothers

```
beer_smooth <- beer |>
  mutate(
    alpha_10 = ses_level(Beer, 0.10),
    alpha_50 = ses_level(Beer, 0.50),
    alpha_90 = ses_level(Beer, 0.90)
  )

beer_smooth |>
  pivot_longer(alpha_10:alpha_90,
    names_to = "alpha", values_to = "smooth") |>
  ggplot(aes(x = Quarter)) +
  geom_line(aes(y = Beer), color = "gray60") +
  geom_line(aes(y = smooth, color = alpha)) +
  labs(y = "Beer production", color = "Smoother")
```

Fit exponential smoothing with fpp3

```
fit_ets <- beer |>  
  model(ETS(Beer))  
  
report(fit_ets)
```

What R is doing

ETS() chooses an exponential smoothing model for error, trend, and seasonality.

Forecast with exponential smoothing

```
fc_ets <- fit_ets |>
  forecast(h = "2 years")

fc_ets |>
  autoplot(beer) +
  labs(y = "Beer production",
       title = "Exponential smoothing forecast")
```

Trend and seasonality extensions

Pattern in the data	Common method
No trend, no seasonality	Simple exponential smoothing
Trend	Holt's method
Trend plus seasonality	Holt-Winters method

Keep the message simple

Exponential smoothing is a family of forecasting methods, not just one formula.

ARIMA as the next thing to learn

Autoregression: the next model idea

An autoregressive model predicts future values using previous values:

$$Y_t = \phi Y_{t-1} + \varepsilon_t.$$

Contrast

Exponential smoothing predicts by smoothing the past. AR models predict by explicitly modeling dependence on previous observations.

Where ARIMA enters

ARIMA combines three ideas:

- **AR**: autoregression, using past values
- **I**: differencing, removing trend or nonstationarity
- **MA**: moving average errors, using past shocks

Bootcamp-level takeaway

ARIMA is important, but it is a next step after understanding forecasting, autocorrelation, training/test data, and benchmark methods.

A minimal ARIMA example

```
fit_arima <- beer |>
  model(ARIMA(Beer))

report(fit_arima)

fit_arima |>
  forecast(h = "2 years") |>
  autoplot(beer)
```

The goal is not to explain every parameter. The goal is to show the workflow.

Simulate different levels of persistence

```
set.seed(1)

low_persistence <- arima.sim(list(ar = 0.2), n = 200)
high_persistence <- arima.sim(list(ar = 0.8), n = 200)
near_random_walk <- arima.sim(list(ar = 0.98), n = 200)

plot(low_persistence, type = "l", main = "phi = 0.2")
plot(high_persistence, type = "l", main = "phi = 0.8")
plot(near_random_walk, type = "l", main = "phi = 0.98")
```

Ask: Which series is easiest to forecast tomorrow? Which is hardest long-term?

Fit a simple ARIMA model

```
fit_arima <- train |>
  model(ARIMA(Beer))

report(fit_arima)

fc_arima <- fit_arima |>
  forecast(h = nrow(test))

fc_arima |>
  autoplot(train) +
  autolayer(test, Beer) +
  labs(title = "ARIMA forecast")
```

ARIMA models use autocorrelation and differencing to forecast.

A gentle explanation of differencing

If a series has a strong trend, model the changes instead of the levels:

$$\Delta Y_t = Y_t - Y_{t-1}.$$

Differencing often turns a wandering nonstationary series into a more stable series.

This connects directly to the random-walk demo:

$$Y_t = Y_{t-1} + \varepsilon_t \quad \Rightarrow \quad \Delta Y_t = \varepsilon_t.$$

Modern workflow checklist

When you see data collected over time:

1. Plot the series.
2. Look for trend.
3. Look for seasonality.
4. Check autocorrelation.
5. Split into training and test periods.
6. Fit simple baselines.
7. Fit time-series models and compare forecast accuracy.

When are time-series methods appropriate?

Use time-series methods when:

- observations are ordered in time;
- nearby observations tend to be related;
- trend or seasonality is part of the signal;
- the goal is forecasting future observations;
- residuals from ordinary regression are autocorrelated.

A time series is not just a dataset with a date column. The date order must matter.

Regression treats observations as exchangeable rows.

Time series treats observations as ordered history.

The past is information.

**And now for something completely
different**

Why include these?

These methods are not time series methods, but they are useful in general data analysis situations.

1. **Bootstrap**: quantify uncertainty by resampling data.
2. **Local linear regression smoothing**: reveal nonlinear relationships.
3. **Kernel density estimation**: estimate the shape of a distribution.

Each one is a practical alternative to relying only on formulas from idealized models.

Bootstrap

Bootstrap: the idea

Suppose we have a sample $x_1, \dots, x_n$ and want uncertainty for a statistic.

Bootstrap logic

If the sample is our best guess of the population, repeatedly resample from the sample and recompute the statistic.

This gives an approximate sampling distribution.

Bootstrap a mean in R

```
set.seed(123)

x <- na.omit(airquality$Ozone)

boot_means <- replicate(2000, {
  x_star <- sample(x, size = length(x), replace = TRUE)
  mean(x_star)
})

quantile(boot_means, c(0.025, 0.975))
```

This is a bootstrap confidence interval for the mean ozone level.

Visualize the bootstrap distribution

```
tibble(boot_mean = boot_means) |>
  ggplot(aes(x = boot_mean)) +
  geom_histogram(bins = 30) +
  labs(x = "Bootstrap means",
       y = "Count",
       title = "Bootstrap sampling distribution")
```

The spread of this distribution represents uncertainty in the estimate.

Local linear regression smoothing

Local smoothing: the idea

Linear regression fits one global line.

Local regression fits many small regressions near each value of x .

Why it helps

It reveals nonlinear patterns without forcing a single polynomial or a single straight line.

To estimate the curve near a point x_0 :

1. Give observations near x_0 high weight.
2. Give observations far from x_0 low weight.
3. Fit a small weighted regression locally.
4. Move x_0 across the plot to trace out a smooth curve.

The bandwidth or span controls how smooth the curve is.

Local regression smoothing in R

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  geom_smooth(method = "loess", se = FALSE) +  
  labs(x = "Weight",  
       y = "Miles per gallon",  
       title = "Local regression smoother")
```

Use this as an exploratory tool, not as automatic proof of a relationship.

Kernel density estimation

Kernel density estimation: the idea

A histogram estimates the shape of a distribution, but it depends heavily on bin width and bin edges.

Kernel density estimation places a small smooth bump at each observation and adds the bumps together.

Main idea

KDE is a smooth version of a histogram.

Bandwidth choice	Result
Too small	Too wiggly
Reasonable	Reveals the distribution shape
Too large	Over-smoothed

The bandwidth plays a role similar to α in smoothing: it controls flexibility.

Kernel density estimation in R

```
ggplot(mtcars, aes(x = mpg)) +  
  geom_histogram(aes(y = after_stat(density)), bins = 10) +  
  geom_density(linewidth = 1) +  
  labs(x = "Miles per gallon",  
        y = "Density",  
        title = "Histogram plus kernel density estimate")
```

We can see the distribution without choosing many arbitrary bins.

Examine a data set from FPP3 and answer three questions:

1. What patterns do you see in the plot?
2. Which forecast would you trust for the next few observations?
3. How would you quantify uncertainty or reveal structure if this were not a time series?